eCOA Solutions

Technical Specification Document

Version	1.0
Date	December 8, 2025
Status	Draft

1. System Overview

1.1 Architecture Overview

The eCOA Solutions platform follows a modern microservices architecture with clear separation between the frontend applications, backend services, and data layer. The system is designed for scalability, security, and regulatory compliance.

High-Level Architecture Components

- **Web Portal:** React-based SPA for sponsor administration
- **Mobile App:** React Native cross-platform app for patients
- **API Gateway:** Kong/AWS API Gateway for routing and rate limiting
- **Backend Services:** Node.js microservices for business logic
- **Database Layer:** PostgreSQL for relational data, MongoDB for form schemas
- **Message Queue:** RabbitMQ/AWS SQS for async processing

2. Technology Stack

Layer	Technology	Justification
Frontend - Web	React 18 + TypeScript	Industry standard, strong typing, large ecosystem
Frontend - Mobile	React Native	Cross-platform, code sharing with web
UI Framework	Tailwind CSS + shadcn/ui	Rapid development, consistent design
State Management	Redux Toolkit + RTK Query	Predictable state, built-in caching
Backend Runtime	Node.js 20 LTS	JavaScript ecosystem, async I/O
Backend Framework	NestJS	Enterprise patterns, TypeScript native
Primary Database	PostgreSQL 15	ACID compliance, JSON support
Document Store	MongoDB 7	Flexible form schemas, fast queries
Cache	Redis 7	Session management, caching
Message Queue	RabbitMQ / AWS SQS	Async processing, notifications
Cloud Platform	AWS / Azure	HIPAA compliant, global regions
Container Orchestration	Kubernetes (EKS/AKS)	Scalability, self-healing
CI/CD	GitHub Actions	Integrated with repo, flexible

3. Microservices Architecture

3.1 Service Decomposition

Service	Responsibility	Database
auth-service	Authentication, authorization, session management	PostgreSQL + Redis
study-service	Study CRUD, template management, cloning	PostgreSQL
form-service	Form builder, schema storage, rendering	MongoDB
submission-service	Form responses, patient data, validation	PostgreSQL + MongoDB
notification-service	Push notifications, email, SMS, scheduling	PostgreSQL + Redis
deployment-service	UAT/Production deployment, versioning	PostgreSQL
report-service	Report generation, analytics, exports	PostgreSQL (read replica)

3.2 Service Communication

- **Synchronous:** REST APIs over HTTPS for real-time operations
- **Asynchronous:** Message queues for notifications, reports, batch processing
- **Event-Driven:** Event bus for cross-service state changes

4. API Specifications

4.1 Authentication APIs

POST /api/v1/auth/login

Request Body:

```
{ "email": "string", "password": "string" }
```

Response:

```
{ "accessToken": "jwt", "refreshToken": "jwt", "user": {...} }
```

POST /api/v1/auth/patient-login

Request Body:

```
{ "patientCode": "string", "studyId": "uuid" }
```

Response:

```
{ "accessToken": "jwt", "pendingForms": [...] }
```

4.2 Study Management APIs

GET /api/v1/studies

Response:

```
{ "studies": [{ "id", "name", "status", "createdAt", "updatedAt" }] }
```

POST /api/v1/studies

Request Body:

```
{ "name": "string", "templateId?": "uuid", "createFromScratch": boolean }
```

POST /api/v1/studies/:id/clone

Response:

```
{ "clonedStudy": { "id", "name", "status": "draft" } }
```

4.3 Form Builder APIs

POST /api/v1/forms

Request Body:

```
{ "studyId": "uuid", "name": "string", "schema": { pages: [...], elements: [...] } }
```

PUT /api/v1/forms/:id/schema

Updates form schema with new elements, pages, and properties.

POST /api/v1/forms/:id/properties

Request Body:

```
{ "completedBy": ["patient"], "notification": {...}, "triggers": {...}, "includeInReports": true }
```

4.4 Submission APIs

POST /api/v1/submissions

Request Body:

```
{ "formId": "uuid", "patientCode": "string", "responses": {...}, "completedAt": "ISO8601" }
```

GET /api/v1/submissions?studyId=:id

Returns all submissions for a study with pagination and filtering.

5. Security Architecture

5.1 Authentication & Authorization

- **JWT Tokens:** Short-lived access tokens (15 min), long-lived refresh tokens (7 days)
- **RBAC:** Role-based access control (Admin, Sponsor, DataManager, Patient)
- **MFA:** Optional multi-factor authentication for sponsors
- **API Keys:** Primary/secondary keys for database access

5.2 Data Protection

- **Encryption at Rest:** AES-256 encryption for all stored data
- **Encryption in Transit:** TLS 1.3 for all communications
- **PII Handling:** Patient codes pseudonymize identity
- **Audit Logging:** Complete audit trail for 21 CFR Part 11 compliance

5.3 Compliance Requirements

1. **21 CFR Part 11:** Electronic records and signatures
2. **HIPAA:** Protected health information safeguards
3. **GDPR:** EU data protection compliance
4. **GxP:** Good practice regulations

6. Infrastructure & DevOps

6.1 Environment Strategy

Environment	Purpose	Infrastructure
Development	Developer testing, feature branches	Shared Kubernetes cluster
Staging	Integration testing, QA	Production-like, scaled down
UAT	Sponsor acceptance testing	Isolated per sponsor/study
Production	Live patient data collection	High availability, multi-AZ

6.2 CI/CD Pipeline

1. **Code Push:** Developer pushes to feature branch
2. **Build:** GitHub Actions builds Docker images
3. **Test:** Unit tests, integration tests, security scans
4. **Deploy to Dev:** Automatic deployment on merge
5. **Deploy to Staging:** Manual promotion with approval
6. **Deploy to UAT:** Per-study isolated deployment
7. **Deploy to Production:** Blue-green deployment with rollback

6.3 Monitoring & Observability

- **Logging:** ELK Stack (Elasticsearch, Logstash, Kibana)
- **Metrics:** Prometheus + Grafana dashboards
- **Tracing:** Jaeger for distributed tracing
- **Alerting:** PagerDuty integration for critical issues
- **APM:** Application performance monitoring

7. Scalability & Performance

7.1 Performance Targets

- API response time: < 200ms (p95)
- Form load time: < 2 seconds
- Concurrent users: 10,000+ per study
- Notification delivery: < 30 seconds
- System uptime: 99.9% SLA

7.2 Scaling Strategy

- **Horizontal Scaling:** Auto-scaling Kubernetes pods based on load
- **Database Scaling:** Read replicas, connection pooling
- **Caching:** Redis for sessions, form schemas, frequent queries
- **CDN:** CloudFront/Azure CDN for static assets

8. Integration Points

- **Push Notifications:** Firebase Cloud Messaging (FCM), Apple Push Notification (APNS)
- **Email Service:** SendGrid / AWS SES

- **SMS Service:** Twilio
- **App Stores:** Apple App Store Connect, Google Play Console APIs
- **Analytics:** Mixpanel / Amplitude for usage analytics